

HisabDo AI — Day 16

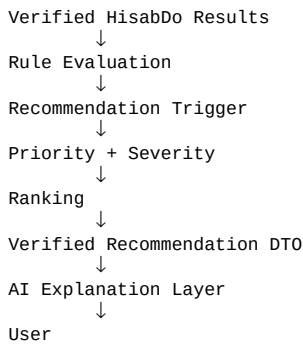
Taha — AI Recommendation Engine: Recommendation Methodology

Detailed Methodology, Rule Logic, Validation, Consistency, Limitations and Future ML Improvements

Objective: Convert verified financial-analysis results into deterministic, explainable recommendations. The recommendation engine must not invent financial facts. HisabDo backend/calculation services remain the source of truth; AI is used later to explain verified recommendations.

1. Recommendation Methodology

The MVP uses a rule-based recommendation methodology. Day 10 Financial Health, Day 11 Spending Patterns, Day 12 Anomalies, Day 13 Cash-Flow Forecast and Day 14 Budget results are treated as verified inputs. Each input is evaluated against configurable conditions. A matching condition creates a recommendation containing category, priority, severity, reason, suggested action and source. The final list is deterministically ranked.



2. Recommendation Categories

Category	Purpose
Saving Improvement	Trigger when verified saving behavior is weak.
Spending Reduction	Address high expense growth or recurring expenses.
Budget Adjustment	Review categories whose verified budget utilization is high.
Cash-Flow Risk	Address verified forecasts showing zero or negative expected saving.
Unusual Spending Review	Review verified unusual transactions/anomalies.
Financial Health Improvement	Address low overall financial-health score.
Category Spending Control	Review categories with significant spending increases.

3. Rule-Based Recommendation Conditions

Trigger	MVP Condition	Recommendation
Low saving rate	Saving Rate < 10%	Improve saving behavior — High
High expense growth	Expense Growth ≥ 20%	Review high-growth spending — High
Category increase	Category change ≥ 20%	Control category spending — Medium
Unusual spending	Backend marks category unusual	Review unusual spending — High
Recurring expense	Backend verifies recurring pattern	Review recurring expense — Medium
Forecast cash-flow risk	Verified Expected Saving ≤ 0	Review projected cash-flow risk — High
Budget high	Utilization > 120%	Review budget — Medium
Budget critical	Utilization ≥ 150%	Review budget — High
Low health	Financial Health Score ≤ 59	Improve financial health — High

These are MVP engineering thresholds, not universal financial advice. They should remain configurable and be validated on real HisabDo datasets.

4. Recommendation Priority Logic

Priority represents how urgently the system should surface a recommendation. High priority is used for conditions that can materially affect the user's near-term financial position in the MVP, such as low saving, high expense growth, low health score, verified unusual spending, or negative/zero forecast saving. Medium priority is used for category increases, recurring expenses and moderate budget over-utilization. Low priority can be used for informational or low-impact recommendations such as budget alignment when no risk trigger is present.

5. Recommendation Severity

Severity describes the magnitude of the underlying verified condition. Low, Medium and High are deterministic labels. The recommendation engine must not let an LLM decide severity. For example, budget utilization $\geq 150\%$ is High severity, while utilization $>120\%$ but below 150% is Medium severity. An anomaly can inherit its verified backend severity.

6. Financial Health Factors as Recommendation Triggers

The engine consumes the verified Financial Health result rather than recalculating it. MVP triggers include Financial Health Score ≤ 59 , Saving Rate $< 10\%$, and Expense Growth $\geq 20\%$. This creates actionable links between Day 10 analysis and Day 16 recommendations. Future versions can inspect individual factor scores directly when those factor values are exposed by the backend.

7. Spending Patterns as Recommendation Triggers

Day 11 provides category-level changes, unusual patterns and recurring-expense candidates. A category increase of at least 20% creates a Category Spending Control recommendation. A verified unusual category creates an Unusual Spending Review recommendation. A verified recurring pattern creates a recurring-expense review recommendation. The engine should use the Day 11 verified flags rather than independently guessing whether a pattern is unusual.

8. Anomalies as Recommendation Triggers

Day 12 anomaly results are treated as verified signals. Any verified anomaly can create an Unusual Spending Review recommendation. The recommendation severity follows the backend anomaly severity, and priority is High for High severity and Medium for lower severities in this MVP. The system should not claim fraud, theft or unauthorized activity unless a trusted source explicitly establishes that fact.

9. Cash-Flow Forecast as Recommendation Trigger

A recommendation is created only when the forecast is marked verified and Expected Saving ≤ 0 . This protects the system from making cash-flow claims when there is insufficient historical data. The recommendation asks the user to review projected expenses and possible actions; it does not invent a future balance.

10. Budget Utilization as Recommendation Trigger

Only verified budget records are evaluated. Utilization above 120% creates a Medium budget review recommendation. Utilization at or above 150% creates a High recommendation. No budget means no over-budget claim. The engine never invents a budget value.

11. Recommendation Ranking Logic

The MVP ranking is deterministic: Priority descending $\rightarrow$ Severity descending $\rightarrow$ Category name $\rightarrow$ Recommendation title. This produces repeatable output for the same verified input. A future version can add deduplication, confidence, recency and impact while keeping the ordering deterministic and auditable.

12. Example Trigger Chains

Example A
Saving Rate Low
+
Expense Growth High
↓
Recommendation Trigger
↓

"Review high-growth spending categories"

Example B

Food Spending ↑

+

Budget Utilization > Threshold

↓

Recommendation

↓

"Review Food spending and budget"

13. Sample Dataset Validation

Verified Input	Value	Expected Output
Financial Health	55/100	Financial Health Improvement — High
Saving Rate	6%	Saving Improvement — High
Expense Growth	25%	Spending Reduction — High
Food category change	28%	Category Spending Control — Medium
Food anomaly	High	Unusual Spending Review — High
Expected saving	-5,000	Cash-Flow Risk — High
Food budget utilization	138%	Budget Adjustment — Medium

14. Measuring Recommendation Consistency

Consistency means the same verified input should produce the same recommendation category, reason, priority, severity and rank. Validation should run the same dataset repeatedly and compare outputs. Recommended checks: deterministic output, threshold-boundary tests (9.99/10/10.01 saving rate; 119.99/120/120.01 utilization; 149.99/150), missing-data behavior, duplicate inputs, ordering stability and user-isolation tests.

15. Assumptions and Limitations

- Thresholds are MVP engineering defaults and must be configurable.
- Recommendations depend on the quality and correctness of upstream Day 10–14 calculations.
- Insufficient history must not be treated as a negative financial signal.
- No budget means no budget-utilization recommendation.
- No verified forecast means no forecast-risk recommendation.
- The engine does not independently calculate source-of-truth financial metrics.
- Recommendations are observations/actions to review, not personalized professional financial advice.
- The AI explanation layer must preserve backend amounts, percentages, dates, categories, scores and recommendation text.

16. Future ML-Based Recommendation Improvements

After the deterministic MVP is validated, ML can learn from historical recommendation outcomes. Possible improvements include personalized thresholds, recommendation impact prediction, ranking based on observed user outcomes, contextual recommendations based on transaction timing, confidence scoring, duplicate recommendation suppression, and feedback-driven adaptation. ML should remain constrained by verified HisabDo data and explicit safety/business rules. It should not be allowed to fabricate financial facts.

17. Reference C# Implementation

```
using System;
using System.Collections.Generic;
using System.Linq;

namespace HisabDo.AI.Recommendations
{
    // Day 16 - Taha: Rule-Based AI Recommendation Engine
    // Golden rule:
    // Backend/calculation engine owns financial facts.
    // This engine converts VERIFIED results into deterministic recommendations.
    // LLM/AI must explain these recommendations; it must not invent or recalculate facts.

    public enum RecommendationCategory
    {
        SavingImprovement,
        SpendingReduction,
        BudgetAdjustment,
        CashFlowRisk,
        UnusualSpendingReview,
        FinancialHealthImprovement,
        CategorySpendingControl
    }

    public enum RecommendationPriority
    {
        Low = 1,
        Medium = 2,
        High = 3
    }

    public enum RecommendationSeverity
    {
        Low = 1,
        Medium = 2,
        High = 3
    }

    public sealed class FinancialHealthInput
    {
        public double Score { get; init; }
        public double SavingRate { get; init; }
        public double ExpenseGrowthPercent { get; init; }
    }

    public sealed class SpendingPatternInput
    {
        public string Category { get; init; } = "";
        public double CurrentAmount { get; init; }
        public double PreviousAmount { get; init; }
        public double ChangePercent { get; init; }
        public double CategorySharePercent { get; init; }
        public bool IsUnusual { get; init; }
        public bool IsRecurring { get; init; }
    }

    public sealed class AnomalyInput
    {
        public string Category { get; init; } = "";
        public string Type { get; init; } = "";
        public string Severity { get; init; } = "Low";
        public double Amount { get; init; }
    }

    public sealed class CashFlowInput
    {
        public double ExpectedIncome { get; init; }
        public double ExpectedExpense { get; init; }
        public double ExpectedSaving { get; init; }
        public bool IsVerifiedForecast { get; init; }
    }

    public sealed class BudgetInput
    {
        public string Category { get; init; } = "";
        public double Budget { get; init; }
        public double ActualExpense { get; init; }
        public double UtilizationPercent { get; init; }
        public bool IsVerified { get; init; }
    }

    public sealed class Recommendation
    {
        public RecommendationCategory Category { get; init; }
        public RecommendationPriority Priority { get; init; }
    }
}
```

```

    public RecommendationSeverity Severity { get; init; }
    public string Title { get; init; } = "";
    public string Reason { get; init; } = "";
    public string SuggestedAction { get; init; } = "";
    public string Source { get; init; } = "";
}

public sealed class RecommendationResult
{
    public string UserId { get; init; } = "";
    public List<Recommendation> Recommendations { get; init; } = new();
}

public sealed class RecommendationInput
{
    public string UserId { get; init; } = "";

    // All inputs below must already be verified by the backend/calculation services.
    public FinancialHealthInput? FinancialHealth { get; init; }
    public List<SpendingPatternInput> SpendingPatterns { get; init; } = new();
    public List<AnomalyInput> Anomalies { get; init; } = new();
    public CashFlowInput? CashFlowForecast { get; init; }
    public List<BudgetInput> Budgets { get; init; } = new();
}

public sealed class RecommendationEngine
{
    // MVP thresholds. Keep configurable in production and validate with real HisabDo data.
    private const double LowSavingRate = 10.0;
    private const double HighExpenseGrowth = 20.0;
    private const double CategoryIncrease = 20.0;
    private const double HighBudgetUtilization = 120.0;
    private const double CriticalBudgetUtilization = 150.0;
    private const double LowHealthScore = 59.0;

    public RecommendationResult Generate(RecommendationInput input)
    {
        if (input == null) throw new ArgumentNullException(nameof(input));
        if (string.IsNullOrEmpty(input.UserId))
            throw new ArgumentException("UserId is required.", nameof(input));

        var result = new List<Recommendation>();

        // 1. Financial Health factors
        if (input.FinancialHealth != null)
        {
            if (input.FinancialHealth.Score <= LowHealthScore)
            {
                result.Add(new Recommendation
                {
                    Category = RecommendationCategory.FinancialHealthImprovement,
                    Priority = RecommendationPriority.High,
                    Severity = RecommendationSeverity.High,
                    Title = "Review overall financial health",
                    Reason = $"Verified financial health score is {input.FinancialHealth.Score:0.##}/100.",
                    SuggestedAction = "Review the main weak financial-health factors and prioritize corrective actions.",
                    Source = "Financial Health"
                });
            }

            if (input.FinancialHealth.SavingRate < LowSavingRate)
            {
                result.Add(new Recommendation
                {
                    Category = RecommendationCategory.SavingImprovement,
                    Priority = RecommendationPriority.High,
                    Severity = RecommendationSeverity.High,
                    Title = "Improve saving behavior",
                    Reason = $"Verified saving rate is {input.FinancialHealth.SavingRate:0.##}%",
                    SuggestedAction = "Review discretionary spending and identify realistic saving opportunities.",
                    Source = "Financial Health"
                });
            }

            if (input.FinancialHealth.ExpenseGrowthPercent >= HighExpenseGrowth)
            {
                result.Add(new Recommendation
                {
                    Category = RecommendationCategory.SpendingReduction,
                    Priority = RecommendationPriority.High,
                    Severity = RecommendationSeverity.High,
                    Title = "Review high-growth spending",
                    Reason = $"Verified expense growth is {input.FinancialHealth.ExpenseGrowthPercent:0.##}%",
                    SuggestedAction = "Review the categories contributing most to expense growth.",
                    Source = "Financial Health"
                });
            }
        }
    }
}

```

```

}

// 2. Spending patterns
foreach (var p in input.SpendingPatterns)
{
    if (p.IsUnusual)
    {
        result.Add(new Recommendation
        {
            Category = RecommendationCategory.UnusualSpendingReview,
            Priority = RecommendationPriority.High,
            Severity = RecommendationSeverity.High,
            Title = $"Review unusual {p.Category} spending",
            Reason = "The backend has verified this category as unusual.",
            SuggestedAction = $"Review recent {p.Category} transactions and confirm whether the spending is expected.",
            Source = "Spending Pattern"
        });
    }

    if (p.IsRecurring)
    {
        result.Add(new Recommendation
        {
            Category = RecommendationCategory.SpendingReduction,
            Priority = RecommendationPriority.Medium,
            Severity = RecommendationSeverity.Medium,
            Title = $"Review recurring {p.Category} expense",
            Reason = "The backend has verified a recurring spending pattern.",
            SuggestedAction = $"Review whether the recurring {p.Category} expense is necessary or can be reduced.",
            Source = "Spending Pattern"
        });
    }

    if (p.ChangePercent >= CategoryIncrease)
    {
        result.Add(new Recommendation
        {
            Category = RecommendationCategory.CategorySpendingControl,
            Priority = RecommendationPriority.Medium,
            Severity = RecommendationSeverity.Medium,
            Title = $"Control {p.Category} spending growth",
            Reason = $"Verified {p.Category} spending increased by {p.ChangePercent:0.##}%",
            SuggestedAction = $"Review what caused the increase in {p.Category} spending.",
            Source = "Spending Pattern"
        });
    }
}

// 3. Anomalies
foreach (var a in input.Anomalies)
{
    var severity = ParseSeverity(a.Severity);
    var priority = severity == RecommendationSeverity.High
        ? RecommendationPriority.High
        : RecommendationPriority.Medium;

    result.Add(new Recommendation
    {
        Category = RecommendationCategory.UnusualSpendingReview,
        Priority = priority,
        Severity = severity,
        Title = $"Review {a.Type} anomaly",
        Reason = $"A verified {a.Type} anomaly was detected in {a.Category}.",
        SuggestedAction = $"Review the related {a.Category} transaction(s) and confirm the activity.",
        Source = "Anomaly Detection"
    });
}

// 4. Cash-flow forecast
if (input.CashFlowForecast?.IsVerifiedForecast == true &&
    input.CashFlowForecast.ExpectedSaving <= 0)
{
    result.Add(new Recommendation
    {
        Category = RecommendationCategory.CashFlowRisk,
        Priority = RecommendationPriority.High,
        Severity = RecommendationSeverity.High,
        Title = "Review projected cash-flow risk",
        Reason = $"Verified forecast shows expected saving of {input.CashFlowForecast.ExpectedSaving:0.##}.",
        SuggestedAction = "Review projected expenses and identify actions that may improve future cash flow.",
        Source = "Cash-Flow Forecast"
    });
}

// 5. Budget utilization
foreach (var b in input.Budgets.Where(x => x.IsVerified))
{

```

```

        if (b.UtilizationPercent >= CriticalBudgetUtilization)
        {
            result.Add(new Recommendation
            {
                Category = RecommendationCategory.BudgetAdjustment,
                Priority = RecommendationPriority.High,
                Severity = RecommendationSeverity.High,
                Title = $"Review {b.Category} budget",
                Reason = $"Verified budget utilization is {b.UtilizationPercent:0.##}%",
                SuggestedAction = $"Review {b.Category} spending and whether the budget needs adjustment.",
                Source = "Budget"
            });
        }
        else if (b.UtilizationPercent > HighBudgetUtilization)
        {
            result.Add(new Recommendation
            {
                Category = RecommendationCategory.BudgetAdjustment,
                Priority = RecommendationPriority.Medium,
                Severity = RecommendationSeverity.Medium,
                Title = $"Review {b.Category} budget",
                Reason = $"Verified budget utilization is {b.UtilizationPercent:0.##}%",
                SuggestedAction = $"Review {b.Category} spending and budget usage.",
                Source = "Budget"
            });
        }
    }

    // 6. Ranking: priority desc, severity desc, then category/title for deterministic output.
    var ranked = result
        .OrderByDescending(x => x.Priority)
        .ThenByDescending(x => x.Severity)
        .ThenBy(x => x.Category.ToString())
        .ThenBy(x => x.Title)
        .ToList();

    return new RecommendationResult
    {
        UserId = input.UserId,
        Recommendations = ranked
    };
}

private static RecommendationSeverity ParseSeverity(string? value)
{
    return value?.Trim().ToLowerInvariant() switch
    {
        "high" => RecommendationSeverity.High,
        "medium" => RecommendationSeverity.Medium,
        _ => RecommendationSeverity.Low
    };
}
}

// Example validation dataset:
// SavingRate=6%, ExpenseGrowth=25%, Food Change=28%,
// Food Budget Utilization=138%, Food anomaly=High,
// ExpectedSaving=-5000.
//
// Expected recommendation categories:
// - SavingImprovement (High)
// - SpendingReduction (High)
// - CategorySpendingControl (Medium)
// - BudgetAdjustment (Medium)
// - UnusualSpendingReview (High)
// - CashFlowRisk (High)
}

```

18. Acceptance Checklist

- ✓ Recommendation methodology documented.
- ✓ Recommendation categories defined.
- ✓ Rule conditions and configurable thresholds implemented.
- ✓ Priority and severity logic implemented.
- ✓ Financial Health, Spending Pattern, Anomaly, Cash-Flow and Budget triggers covered.
- ✓ Deterministic ranking implemented.
- ✓ Sample dataset validation defined.

- ✓ Consistency/boundary testing defined.
- ✓ Assumptions and limitations documented.
- ✓ Future ML roadmap documented.
- ✓ Verified-data-only boundary preserved.
- ✓ C# implementation supplied.